

WTB & WTS Hosting, Selling, Distribution, and Security Options

This document outlines practical options to host, sell, and distribute WTB & WTS without running a server at home, plus security guidance for a paid Chrome extension.

Constraints and goals

- Do not self-host from home.
- Sell through a reliable payment flow.
- Distribute primarily through Chrome Web Store.
- Reduce copying abuse while accepting that client code can be extracted.

1. Hosting options (no home server)

Option A: Serverless backend (recommended starting point)

Use hosted serverless functions plus managed storage.

Examples:

- Cloudflare Workers + D1 + KV/R2
- Vercel Functions + Neon/Supabase Postgres + Blob storage
- AWS Lambda + API Gateway + DynamoDB + S3

Use cases:

- License verification endpoint
- Entitlement checks for premium features
- Usage/rate-limit enforcement

Pros:

- No server maintenance
- Scales automatically
- Pay for usage

Cons:

- Vendor-specific limits
- Requires good monitoring and retries

Option B: Backend-as-a-Service (BaaS)

Use a hosted platform with auth and database included.

Examples:

- Supabase

- Firebase

Use cases:

- User auth and session management
- License state storage
- Admin dashboard for entitlements

Pros:

- Fast to ship
- Built-in auth and data APIs

Cons:

- Security rules must be carefully configured
- Vendor lock-in risk

Option C: Managed app hosting (PaaS)

Run a normal API service on a managed host.

Examples:

- Render
- Railway
- Fly.io

Pros:

- Full control of API framework/runtime

Cons:

- More ops than serverless/BaaS
- Must manage scaling and uptime settings

2. Selling options

Merchant of Record platforms (simplest for tax/compliance)

Examples:

- Paddle
- Lemon Squeezy

Pros:

- Handles VAT/sales tax and many compliance tasks
- Quick checkout setup

Cons:

- Platform fees and payout schedule constraints

Direct processor

Example:

- Stripe

Pros:

- Full checkout control
- Broad ecosystem

Cons:

- You handle tax stack, invoicing rules, and more legal overhead

3. Distribution options

Primary: Chrome Web Store

- Best install trust and update flow for Chrome users.
- Required for mainstream reach.

Secondary: Edge Add-ons

- Minimal extra work for more users.
- Same client code exposure risk as Chrome.

Avoid for paid consumer workflow

- Direct CRX distribution for general users.
- Manual update channels that increase support burden.

4. Security model for paid extensions

What cannot be fully protected

- Client-side extension code can be unpacked and inspected.
- Obfuscation slows reverse engineering but does not stop it.

What you can protect effectively

- Paid feature access
- License validity
- Entitlement decisions

Recommended controls

- Keep license checks and entitlement logic server-side.
- Use short-lived signed tokens (for example, 5 to 30 minutes).
- Revalidate entitlements periodically and on startup.
- Add rate limits per account/license/IP.
- Add abuse detection and revoke compromised keys quickly.

- Remove source maps and debug helpers from release builds.
- Minify/obfuscate release JS as a deterrent.

5. Legal and privacy assets to publish

Required project assets:

- Privacy Policy
- Terms of Use / EULA
- Refund policy
- Non-affiliation disclaimer
- Support and legal contact email

Suggested minimum disclosures:

- What data is stored locally/synced
- What data is sent to third-party APIs
- Whether personal data is sold (usually no)
- Retention/deletion process
- How paid-license data is processed

6. Suggested rollout plan (no home server)

Phase 1: Foundation (1 week)

- Pick payment platform (Paddle/Lemon Squeezy/Stripe).
- Publish Privacy Policy and Terms drafts.
- Add legal disclaimers in extension and store listing.

Phase 2: License API MVP (1 to 2 weeks)

- Build endpoints: activate, validate, deactivate license.
- Store license state and device/session limits.
- Add logging, rate limits, and admin revoke action.

Phase 3: Extension integration (1 week)

- Add startup entitlement check and periodic refresh.
- Gate premium features behind server-validated status.
- Add graceful offline behavior and clear user messaging.

Phase 4: Hardening and launch prep (1 week)

- Minify/obfuscate release assets.
- Remove debug logs/source maps.
- Run abuse test cases (shared keys, replay, expired tokens).

7. Recommended stack for your case

If you want low ops and fast launch:

- Hosting: Cloudflare Workers + D1 (or Supabase)
- Selling: Paddle or Lemon Squeezy
- Distribution: Chrome Web Store (+ Edge Add-ons later)
- Security: server-side entitlement checks + short-lived signed tokens + revocation

This gives strong business protection without home hosting and without heavy infrastructure overhead.